



Estructuras de Datos y Algoritmos
Equipo AFIC (ayudantias.fic@org.estudiantilesudp.cl)
Diego Banda(diego.banda@mail.udp.cl)

1. Para las siguientes funciones determine las cotas superiores utilizando notación Big O.

- (a) $5n + 3$
- (b) $4n^2 + 100n + 25$
- (c) $2^n + n^3$
- (d) $\log n + 100$
- (e) $3n^3 + 2n \log n + 7$
- (f) $n! + 2^n$
- (g) $\sqrt{n} + \log n$
- (h) $2^n + 3^n$
- (i) $n + n^2 + n^3 + \dots n^k$
- (j) $\frac{n^3}{1000} + 999n$

2. Para los siguientes algoritmos, analice el tiempo de ejecución en términos de Big O ($O(f(n))$).

(a)

```
public static void f1(int N){
    for(int i = 0 ; i < N ; i++){
        // Body
    }
}
```

(b)

```
public static void f2(){
    for(int i = 0 ; i < 1024 ; i++){
        // Body
    }
}
```

(c)

```
public static void f3(int N){
    for(int i = 0 ; i < N*N ; i++){
        // Body
    }
}
```

(d) Tiempo de ejecución del método $f5$

```
public static void f4(int N){
    for(int i = 0 ; i < N ; i++){
        System.out.println(i);
    }
}

public static void f5(){
    for(int i = 0 ; i < 1000 ; i++){
        f4(i);
    }
}
```



```
(e) public static int f6(int N){  
    if(N == 0) return 0;  
    return f6(N-1) + 1;  
}
```

```
(f) public static void f7(int N){  
    if(N == 0) return 0;  
    f7(N - 1);  
    f7(N - 1);  
}
```

```
(g) public static int f6(int N){  
    if(N == 0) return 0;  
    return f6(N / 2) + 1;  
}
```

3. Usted recibe una carta con un algoritmo misterioso en ella. Dado que usted es un experto en la materia, responderá las siguientes preguntas acerca de este misterioso algoritmo:

```
static int AlgoritmoMisterioso(String expr){  
    Stack <Integer> stack = new Stack<>();  
  
    for (int i = 0 ; i < expr.length() ; i++){  
        char c = expr.charAt(i);  
  
        if (c == ' '){  
            continue;  
        }  
  
        if (Character.isDigit(c)){  
            stack.push(c - '0');  
        }  
        else {  
            int v1 = stack.pop();  
            int v2 = stack.pop();  
  
            switch (c){  
                case '+':  
                    stack.push(v2 + v1);  
                    break;  
                case '-':  
                    stack.push(v2 - v1);  
                    break;  
                case '/':  
                    stack.push(v2 / v1);  
                    break;  
                case '*':  
                    stack.push(v2 * v1);  
                    break;  
            }  
        }  
    }  
}
```



```
    }  
    return stack.pop();  
}
```

Pista: La expresión `c - '0'` tiene como resultado el valor del número `c`. Ejemplos: `'0' - '0' = 0`, `'4' - '0' = 4`.

- (a) Para cada uno de los siguientes casos de prueba, indique el valor retornado por el método *AlgoritmoMisterioso* al recibir el caso como argumento:
- `5 5 +`
 - `6 6 / 9 *`
 - `2 4 * 2 2 * +`
- (b) En base a sus observaciones del punto anterior, explique en no más de **3 líneas**, ¿De qué forma el algoritmo modifica los datos que recibe?
- (c) Analice el algoritmo en cuestión y describa su tiempo de ejecución en términos de **$O(f(n))$** , dónde **n** es el largo del arreglo recibido como argumento y **f** es una función matemática propuesta por usted. Fundamente su respuesta.

4. Se define una lista enlazada de la siguiente manera:

```
class linkedList {  
    class Node {  
        int value;  
        Node next;  
    }  
    Node head;  
}
```

Analice con detenimiento el siguiente algoritmo y responda las siguientes preguntas.

```
static Node algoritmoMisterioso(Node head) {  
    if(head == null || head.next == null)  
        return head;  
  
    Node temp = algoritmoMisterioso(head.next);  
  
    head.next.next = head;  
  
    head.next = null;  
  
    return temp;  
}
```

- (a) Calcule el output del algoritmo para la siguiente lista enlazada.
- LL: `1 -> 7 -> 10 -> 8 -> 666 -> null`
- (b) Describa la complejidad de tiempo del algoritmo utilizando **$O(f(n))$** , argumente su respuesta de manera breve.



*“Simplicity is a great virtue but it requires
hard work to achieve it and education to
appreciate it. And to make matters worse:
complexity sells better”*

— Edsger Dijkstra.